

Compiladores: Panorama General y Contexto

Antecedentes y Relevancia de los Compiladores

M.C. MELISSA BRIGITTHE HINOJOSA CABELLO

Universidad Nacional Autónoma de México

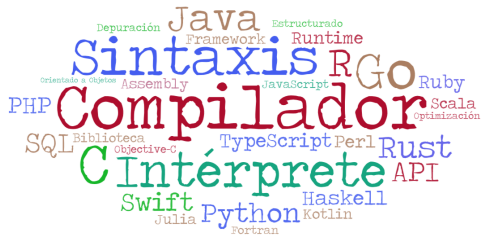
Facultad de Ingeniería

18 de Agosto de 2026





Antes de hablar de compiladores, ¿qué es un lenguaje de programación?



Lenguaje de programación

Es una herramienta que permite desarrollar software, sirviendo como ***una interfaz*** entre el programador y la computadora.

Los lenguajes de programación son lenguajes formales que se conforman de una serie de ***símbolos*** y reglas de ***sintaxis*** y ***semántica*** que definen la estructura principal del lenguaje y le dan un significado a sus elementos y expresiones.



Entonces, un lenguaje de programación:

- Consta de símbolos y reglas precisas (lenguaje formal)

Símbolos

Letras: a–z, A–Z

Dígitos: 0–9

Tokens

Palabras clave:

if, while, return, class, ...

Operadores:

*+, −, *, /, <, >, =, ==*

Delimitadores:

() { } ;

Reglas

Reglas léxicas:

identificador $\rightarrow$ letra (letra | dígito)* (e.g., *var1*)

Reglas sintácticas:

<asignación> $\rightarrow$ identificador = <expresión> ;
(e.g., *var1 = 10*)

Reglas semánticas:

La evaluación de una expresión aritmética produce un valor (e.g., *var1 = 10 + 2*)



Entonces, un lenguaje de programación:

- Tiene secuencias válidas de símbolos (sintaxis)

La sintaxis define **cómo deben ordenarse los símbolos** para que el programa esté bien escrito, i.e., respetando la estructura del lenguaje. Ejemplo: ***var1* = 10 + 2;**

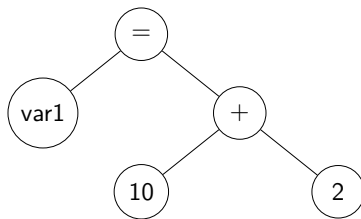
Regla 1. <asignación> → identificador = <expresión> ;

Regla 2. <expresión> → número + número | número

Consta de:

- identificador (var1)
- operador (=)
- expresión válida (10 + 2)
- delimitador (;)

Representación:





Entonces, un lenguaje de programación:

- Define qué hace un programa (semántica)

La semántica define **el significado** del programa, explicando **el efecto** de cada instrucción en el programa cuando ésta se ejecuta. Ejemplo: ***var1 = 10 + 2;***

Explicaciones

- La evaluación de una expresión aritmética produce un valor.
- La expresión $10 + 2$ se evalúa a 12.
- El valor 12 se asigna a *var1*.

Sintaxis → ¿está bien escrito?

vs.

Semántica → ¿qué hace al ejecutarse?



Mediante los lenguajes de programación se expresan instrucciones de manera comprensible para los humanos; el “problema” es que las computadoras solo pueden ejecutar instrucciones en lenguaje máquina.

Lenguaje de alto nivel:

$\text{var1} = 10 + 2;$

Lenguaje de bajo nivel:

Ensamblador

LOAD 10 ; *cargar 10 en un registro*

ADD 2 ; *sumar 2*

STORE var1 ; *guardar el resultado en var1*

Ensamblador basado en registros

MOV R1, #10 ; *R1 toma el valor 10*

ADD R1, R1, #2 ; *R1 = R1 + 2*

MOV var1, R1 ; *var1 toma el valor de R1*



Es necesario convertir las instrucciones escritas en lenguaje de alto nivel a instrucciones en binario. El lenguaje ensamblador es una representación simbólica del lenguaje máquina y se traduce de forma casi directa a binario.

Código fuente $\longrightarrow$ ¿? $\longrightarrow$ Binario

Para que un programa pueda ejecutarse, el código fuente debe ser transformado de manera sistemática por ***“programas especiales”*** hasta obtener instrucciones en lenguaje máquina, de forma que éstas puedan ser entendidas y ejecutadas por la computadora.

¿Qué es un compilador y por qué es importante?



Esos “programas especiales” convierten las instrucciones escritas en código fuente, en instrucciones escritas en lenguaje máquina (0 y 1).

Compilador

Es un sistema de software que procesa programas escritos en lenguajes de alto nivel en una representación equivalente de instrucciones que puedan ser ejecutadas por la computadora.

Para ello debe:

- ▶ Preservar el significado del programa.
- ▶ Identificar y reportar errores en el programa.
- ▶ Optimizar o generar código lo más eficiente posible.



¿Por qué no solo programamos directamente en lenguaje máquina?

Porque programar directamente
en lenguaje máquina **no es viable**.

- ▶ Extrema dificultad de lectura y escritura.
- ▶ Alta propensión a errores.
- ▶ Compatibilidad y/o portabilidad nula.
- ▶ Difícil mantenimiento y evolución.
- ▶ Baja productividad.



Por eso los programas se escriben empleando lenguajes de alto nivel (e.g., C, Java, Python) porque son legibles y permiten abstraer complejidad.

→ Pero las computadoras solo siguen entendiendo lenguaje máquina.

Por eso es necesario un mecanismo de traducción sistemática y confiable que sea capaz de:

- Comprender la estructura del programa e identificar errores (lenguaje fuente).
- Traducir correctamente (lenguaje destino).
- Generar, además, instrucciones eficientes.

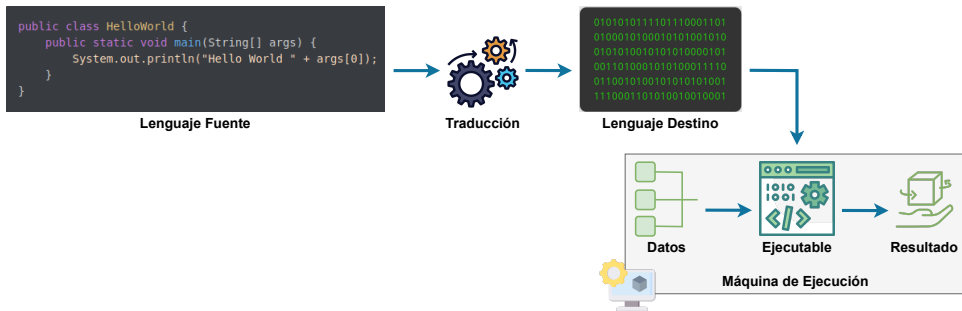


En este sentido, los compiladores resuelven todos estos problemas, ya que proveen:

- ▶ **Abstracción de la arquitectura.** No es necesario conocer los detalles del hardware.
- ▶ **Consistencia y confiabilidad.** Un programa siempre producirá el mismo resultado en diversas ejecuciones.
- ▶ **Portabilidad, reutilización y modularidad.** Un mismo programa puede ejecutarse en diferentes arquitecturas, aprovechando las librerías, módulos y funciones disponibles.
- ▶ **Preparación para optimización.** Se puede mejorar el rendimiento de manera automática eliminando código muerto, reorganizando instrucciones, etc.
- ▶ **Detección de errores.** Se detectan inconsistencias lógicas, tipos incorrectos o variables no inicializadas para evitar errores en tiempo de ejecución.

Compilador

Es un sistema de software que procesa programas escritos en un **lenguaje fuente** en una representación equivalente de instrucciones, **en un lenguaje destino**, que puedan ser ejecutadas por la computadora (**máquina de ejecución**).





- ▶ **Lenguaje fuente:** Programa escrito por el programador en un lenguaje de alto nivel.
- ▶ **Lenguaje destino:** Representación del programa que puede ser ejecutada por la computadora.
- ▶ **Máquina de ejecución:** Hardware que ejecuta las instrucciones en lenguaje máquina.
 - **Arquitectura de hardware.** La ejecución es directa sobre procesadores físicos (CPU, GPU).
 - **Máquinas virtuales.** Simulan una máquina sobre arquitecturas de procesamiento físicas.
 - **Máquinas abstractas.** Definen cómo se ejecutan las instrucciones, sin comprometerse con un hardware específico.
 - **Máquinas interpretadas.** La ejecución ocurre instrucción por instrucción, mediada por un intérprete que decide en tiempo de ejecución qué acciones realizar.
 - **Máquinas híbridas.** Es un entorno de ejecución híbrido que interpreta instrucciones, genera código máquina en tiempo de ejecución y gestiona la ejecución sobre el hardware.

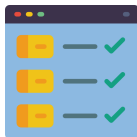
¿Qué debe hacer un compilador para cumplir su objetivo?



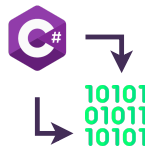
Para transformar un programa en un ejecutable, el compilador cumple distintas funciones, cada una con un propósito específico. Aunque los roles del compilador no son etapas secuenciales, existe una dependencia lógica: primero se analiza y verifica el programa; posteriormente se traduce y optimiza su representación.



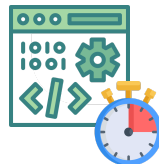
Analizador



Verificador



Traductor



Optimizador



Tarea 1: Historia de los lenguajes y traductores

→ Investigar acerca de:

- Primeros programas y la necesidad de traductores
- Aparición de los lenguajes de alto nivel
- Evolución de los lenguajes y su impacto
- Evolución de los compiladores

→ Elaborar un reporte con un resumen de los puntos anteriores.

→ Entrega: **Jueves 20/Agosto/2026** (antes de la clase).

- Enviar reporte (digital) por correo.
- Discusión en clase.